LATIHAN TERSTRUKTUR

Tugas Pertemuan 4:

Implementasi Unit Testing dengan Github Actions Implementasi Unit Testing dengan GitHub Actions

Nama	:	Melanie Priatna
NIM	:	3224015
Prodi	:	Sistem Informasi
Matkul	:	Implementasi Pengujian Perangkat Lunak
Tanggal	:	12 Oktober 2026
User Git-Hub	:	MelaniePriatna24

BAGIAN A: GIT & GITHUB DASAR

Soal 1

1. Perbedaan Centralized VCS dan Distributed VCS

Version Control System (VCS) adalah sistem yang mencatat setiap perubahan pada file dari waktu ke waktu sehingga versi tertentu bisa dipanggil lagi kapan saja. Berdasarkan arsitekturnya, VCS dibagi menjadi dua jenis, yaitu terpusat (centralized) dan terdistribusi (distributed). Perbedaannya saya rangkum pada tabel berikut.

Aspek	Centralized VCS (CVCS)	Distributed VCS (DVCS)
Penyimpanan	Seluruh versi disimpan di satu server pusat. Developer hanya mengambil (checkout) versi yang sedang dikerjakan.	Setiap developer memiliki salinan lengkap repository beserta seluruh history di komputernya masing-masing.
Kerja offline	Tidak bisa, karena commit harus terhubung ke server pusat.	Bisa, commit dan lihat history dilakukan di lokal, baru dikirim ke remote saat online.
Risiko kegagalan	Single point of failure. Jika server pusat mati atau datanya rusak, semua orang terhenti.	Tidak ada single point of failure. Salinan di mesin developer bisa dipakai memulihkan repository.
Branching	Relatif lebih berat dan kurang praktis.	Lebih mudah dan cepat sehingga cocok untuk pengembangan fitur paralel.
Contoh	Subversion (SVN) dan CVS (selain itu ada Perforce)	Git dan Mercurial

Jadi, dua contoh untuk masing-masing jenis adalah: Centralized VCS: SVN dan CVS; Distributed VCS: Git dan Mercurial. Git menjadi yang paling populer saat ini karena dapat dipakai offline dan branching-nya mudah, serta menjadi dasar integrasi dengan GitHub Actions.

2. Lima Git command yang wajib dikuasai

Command	Fungsi	Contoh
`git add`	Menambahkan perubahan file ke staging area sebelum di-commit.	`git add .`
`git commit`	Menyimpan perubahan yang ada di staging area ke history repository lokal beserta pesan commit.	`git commit -m "feat: add unit test"`
`git push`	Mengirim commit dari repository lokal ke remote repository (GitHub). Push inilah yang memicu workflow GitHub Actions.	`git push origin main`
`git pull`	Mengambil perubahan terbaru dari remote dan menggabungkannya ke repository lokal.	`git pull origin main`
`git branch`	Melihat daftar branch atau membuat branch baru untuk mengerjakan fitur secara terpisah dari main.	`git branch feature-login`

Selain lima command di atas, ada `git init` (membuat repository baru), `git checkout` (pindah branch), `git merge` (menggabungkan branch), dan `git log` (melihat history commit). Command-command tersebut juga saya pakai pada Soal 2.

3. Fungsi .gitignore

`.gitignore` adalah file teks di root repository yang berisi daftar file atau folder yang tidak perlu dilacak dan tidak boleh ikut di-commit oleh Git. Fungsi utamanya adalah menjaga repository tetap bersih, yaitu mencegah file hasil build, file konfigurasi IDE, file sistem operasi, dan file sensitif (misalnya password atau API key) ikut masuk ke repository. Dengan begitu ukuran repository tidak membengkak dan tidak terjadi konflik akibat file yang berbeda di tiap komputer anggota tim. Contoh isi `.gitignore` untuk project Maven:

```
target/         # hasil build Maven
.idea/          # konfigurasi IntelliJ
*.iml
.vscode/
.DS_Store       # file sistem macOS
Thumbs.db       # file sistem Windows
*.exec         # data coverage JaCoCo
```

Soal 2

Praktik dijalankan menggunakan Git Bash. Placeholder yang berwarna kuning menyesuaikan akun GitHub dan NIM saya. Command yang digunakan adalah sebagai berikut.

```
# 1. Buat repo di GitHub (via web): New repository > latihan-git-[NIM]
#    (opsional lewat GitHub CLI)
gh repo create latihan-git-[NIM] --public

# 2. Clone repository ke lokal
git clone https://github.com/[username]/latihan-git-[NIM].git
cd latihan-git-[NIM]

# 3. Buat README.md berisi nama dan NIM
echo "# latihan-git-[NIM]" > README.md
echo "Nama : [Nama Lengkap]" >> README.md
echo "NIM  : [NIM]" >> README.md

# 4. Commit file README.md
git add README.md
git commit -m "docs: add README"

# 5. Push ke GitHub
git branch -M main
git push -u origin main
```

```
# 6. Buat branch baru feature-test
git branch feature-test

# 7. Pindah ke branch feature-test
git checkout feature-test

# 8. Buat test.txt lalu commit
echo "ini file test" > test.txt
git add test.txt
git commit -m "test: add test file"

# 9. Push branch feature-test ke GitHub
git push -u origin feature-test

# 10. Merge feature-test ke main
git checkout main
git merge feature-test
git push origin main

# (verifikasi riwayat commit)
git log --oneline
```

Penjelasan singkat tiap langkah:

No	Command inti	Keterangan
1	`gh repo create` / New repository di web	Membuat repository kosong bernama `latihan-git-[NIM]` (tanpa mencentang opsi "Add a README" agar tidak bentrok dengan langkah 3).
2	`git clone <url>`	Menyalin repository dari GitHub ke komputer lokal, lalu masuk ke foldernya dengan `cd`.
3	`echo ... > README.md`	Membuat README.md berisi nama dan NIM. Tanda > membuat file baru, >> menambah baris.

No	Command inti	Keterangan
4	`git add + git commit -m "docs: add README"`	File dimasukkan ke staging area, lalu disimpan sebagai commit dengan pesan sesuai permintaan soal.
5	`git push -u origin main`	Mengirim commit ke GitHub. `git branch -M main` memastikan nama branch lokal adalah main. Opsi `-u` menghubungkan branch lokal dengan remote.
6	`git branch feature-test`	Membuat branch baru dari posisi commit terakhir di main.
7	`git checkout feature-test`	Berpindah ke branch feature-test (alternatif: `git switch feature-test`).
8	`git add + git commit -m "test: add test file"`	Membuat test.txt dan meng-commit-nya, commit ini hanya ada di branch feature-test.
9	`git push -u origin feature-test`	Mengirim branch feature-test ke GitHub.
10	`git checkout main` lalu `git merge feature-test`	Kembali ke main kemudian menggabungkan feature-test ke dalamnya. Karena main belum berubah sejak branch dibuat, merge berjalan sebagai fast-forward. Terakhir main di-push agar GitHub ikut terbarui.

BAGIAN B: UNIT TESTING FRAMEWORK

Soal 3

Kode yang diuji berasal dari Tugas Minggu 3 (Soal 3 – Validasi Pendaftaran). Agar bisa dijalankan oleh JUnit, method tersebut dibungkus dalam class `PendaftaranService` pada package `com.stmik` (isi method tidak diubah).

PendaftaranService.java

```
package com.stmik;

public class PendaftaranService {

    public String validasiPendaftaran(String nama, int umur,
                                      String email, boolean
setujuSyarat) {
        if (nama == null || nama.isEmpty()) {          // P1
            return "Nama harus diisi";
        }

        if (umur < 17) {                               // P2
            return "Umur minimal 17 tahun";
        }

        if (email == null || !email.contains("@")) {   // P3,
P4
            return "Email tidak valid";
        }

        if (!setujuSyarat) {                           // P5
            return "Harus menyetujui syarat dan ketentuan";
        }

        return "Pendaftaran berhasil";
    }
}
```

Dasar: Flow Graph, Cyclomatic Complexity, dan Independent Path

Setiap pernyataan `if` pada kode merupakan satu predicate node (simpul keputusan). Kondisi gabungan seperti `nama == null || nama.isEmpty()` diperlakukan sebagai satu keputusan karena keduanya menghasilkan return yang sama (label P3 dan P4 pada email dijelaskan di bagian bawah). Penomoran simpul flow graph adalah sebagai berikut.

Simpul	Isi
1	`if (nama == null || nama.isEmpty())` (P1)
2	`return "Nama harus diisi"`
3	`if (umur < 17)` (P2)
4	`return "Umur minimal 17 tahun"`
5	`if (email == null || !email.contains("@"))` (P3, P4)
6	`return "Email tidak valid"`
7	`if (!setujuSyarat)` (P5)
8	`return "Harus menyetujui syarat dan ketentuan"`
9	`return "Pendaftaran berhasil"`
10	Simpul keluar (exit)

Jumlah simpul N = 10 dan jumlah edge E = 13 (8 edge dari simpul keputusan, ditambah 5 edge dari simpul return ke exit). Predicate node ada 4 (simpul 1, 3, 5, 7). Cyclomatic Complexity dihitung dengan tiga rumus dari materi:

- V(G) = E − N + 2 = 13 − 10 + 2 = 5
- V(G) = P + 1 = 4 + 1 = 5
- V(G) = jumlah region = 5

Karena V(G) = 5, terdapat 5 independent path yang harus diuji, yaitu:

- Path 1: 1 – 2 – 10 (nama kosong)
- Path 2: 1 – 3 – 4 – 10 (umur kurang dari 17)

- Path 3: 1 – 3 – 5 – 6 – 10 (email tidak valid)
- Path 4: 1 – 3 – 5 – 7 – 8 – 10 (tidak menyetujui syarat)
- Path 5: 1 – 3 – 5 – 7 – 9 – 10 (pendaftaran berhasil)

1. Lima test case berdasarkan independent path

ID	Path	Input (nama, umur, email, setujuSyarat)	Predicate	Expected output
TC1	Path 1	"" , 20, "budi@mail.com", true	P1 = true	Nama harus diisi
TC2	Path 2	"Budi", 16, "budi@mail.com", true	P1 = false P2 = true	Umur minimal 17 tahun
TC3	Path 3	"Budi", 20, "budi.mail.com", true	P1, P2 = false P3/P4 = true	Email tidak valid
TC4	Path 4	"Budi", 20, "budi@mail.com", false	P1–P4 = false P5 = true	Harus menyetujui syarat dan ketentuan
TC5	Path 5	"Budi", 20, "budi@mail.com", true	P1–P5 = false	Pendaftaran berhasil

Catatan: pada kondisi gabungan, tiap sub-kondisi (nama `null`, email `null`) juga perlu diuji agar lebih teliti. Karena itu pada kode JUnit saya tambahkan tiga test tambahan di luar lima test utama, yaitu nama `null`, email `null`, dan nilai batas umur 17. Tambahan ini tidak mengubah jumlah independent path.

2. Implementasi unit test JUnit

Setiap path dipetakan ke satu method `@Test` dengan pola Arrange-Act-Assert seperti yang diajarkan di materi. Objek diinisialisasi di `@BeforeEach` agar setiap test berdiri sendiri.

PendaftaranServiceTest.java

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class PendaftaranServiceTest {

    private PendaftaranService service;

    @BeforeEach
    public void setUp() {
        service = new PendaftaranService();
    }

    // ---------- 5 test case utama (1 path = 1 test) -------
---

    @Test
    public void testPath1_NamaKosong() {
        // Arrange
        String nama = "";
        // Act
        String result = service.validasiPendaftaran(
                nama, 20, "budi@mail.com", true);
        // Assert
        assertEquals("Nama harus diisi", result);
    }
```

```
    @Test
    public void testPath2_UmurKurangDari17() {
        // Arrange
        int umur = 16;
        // Act
        String result = service.validasiPendaftaran(
                "Budi", umur, "budi@mail.com", true);
        // Assert
        assertEquals("Umur minimal 17 tahun", result);
    }

    @Test
    public void testPath3_EmailTidakValid() {
        // Arrange
        String email = "budi.mail.com";   // tidak ada
karakter @
        // Act
        String result = service.validasiPendaftaran(
                "Budi", 20, email, true);
        // Assert
        assertEquals("Email tidak valid", result);
    }

    @Test
    public void testPath4_TidakSetujuSyarat() {
        // Arrange
        boolean setuju = false;
        // Act
        String result = service.validasiPendaftaran(
                "Budi", 20, "budi@mail.com", setuju);
        // Assert
        assertEquals("Harus menyetujui syarat dan ketentuan",
result);
    }
```

```
    @Test
    public void testPath5_PendaftaranBerhasil() {
        // Act
        String result = service.validasiPendaftaran(
                "Budi", 20, "budi@mail.com", true);
        // Assert
        assertEquals("Pendaftaran berhasil", result);
    }

    // ------ test tambahan untuk sub-kondisi (condition
coverage) ------

    @Test
    public void testTambahan_NamaNull() {
        assertEquals("Nama harus diisi",
                service.validasiPendaftaran(null, 20,
"budi@mail.com", true));
    }

    @Test
    public void testTambahan_EmailNull() {
        assertEquals("Email tidak valid",
                service.validasiPendaftaran("Budi", 20, null,
true));
    }

    @Test
    public void testTambahan_BatasUmur17() {
        assertEquals("Pendaftaran berhasil",
                service.validasiPendaftaran("Budi", 17,
"budi@mail.com", true));
    }
}
```

Soal 4

1. Cyclomatic Complexity calculateGrade()

Penomoran simpul pada flow graph method `calculateGrade()`:

Simpul	Isi
1	`if (score < 0 || score > 100)`
2	`throw new IllegalArgumentException(...)`
3	`if (score >= 80)`
4	`return "A"`
5	`else if (score >= 70)`
6	`return "B"`
7	`else if (score >= 60)`
8	`return "C"`
9	`else if (score >= 50)`
10	`return "D"`
11	`return "E"` (else terakhir)
12	Simpul keluar (exit)

Edge: 1→2, 1→3, 3→4, 3→5, 5→6, 5→7, 7→8, 7→9, 9→10, 9→11 (10 edge), ditambah 2→12, 4→12, 6→12, 8→12, 10→12, 11→12 (6 edge), sehingga E = 16 dan N = 12. Predicate node: simpul 1, 3, 5, 7, 9 (P = 5).

- V(G) = E − N + 2 = 16 − 12 + 2 = 6
- V(G) = P + 1 = 5 + 1 = 6
- V(G) = jumlah region = 6

Cyclomatic Complexity calculateGrade() = 6. (Apabila kondisi gabungan `score < 0 || score > 100` dihitung sebagai dua predicate terpisah, nilainya menjadi 7, tetapi hasil pengujiannya tetap sama karena kedua sub-kondisi tersebut sudah saya uji.)

2. Independent Path calculateGrade()

Path	Jalur simpul	Kondisi	Hasil	Contoh input
1	1 – 2 – 12	score < 0 atau score > 100	Exception	-5 / 101
2	1 – 3 – 4 – 12	score ≥ 80	"A"	85
3	1 – 3 – 5 – 6 – 12	70 ≤ score < 80	"B"	75
4	1 – 3 – 5 – 7 – 8 – 12	60 ≤ score < 70	"C"	65
5	1 – 3 – 5 – 7 – 9 – 10 – 12	50 ≤ score < 60	"D"	55
6	1 – 3 – 5 – 7 – 9 – 11 – 12	score < 50 (dan ≥ 0)	"E"	30

Jumlah path (6) sama dengan nilai V(G), sehingga minimal dibutuhkan 6 test case untuk `calculateGrade()`.

3. Implementasi unit test JUnit

Test untuk `calculateGrade()` dibuat per independent path (path 1 dipisah menjadi nilai negatif dan di atas 100), ditambah pengujian nilai batas (0, 49, 50, 59, 60, 69, 70, 79, 80, 100) karena kesalahan operator pembanding (>= tertulis >) paling sering muncul di batas. Method `isPassed()` diuji pada batas kelulusan 60 serta nilai di luar rentang.

StudentGradeTest.java

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

    private StudentGrade studentGrade;

    @BeforeEach
```

```
    public void setUp() {
        studentGrade = new StudentGrade();
    }

    // ===== calculateGrade(): 1 test per independent path
=====

    @Test
    public void testCalculateGrade_Path1a_ScoreNegatif() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(-5));
        assertEquals("Score must be between 0 and 100",
ex.getMessage());
    }

    @Test
    public void testCalculateGrade_Path1b_ScoreLebihDari100()
{
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(101));
        assertEquals("Score must be between 0 and 100",
ex.getMessage());
    }

    @Test
    public void testCalculateGrade_Path2_NilaiA() {
        assertEquals("A", studentGrade.calculateGrade(85));
    }

    @Test
    public void testCalculateGrade_Path3_NilaiB() {
        assertEquals("B", studentGrade.calculateGrade(75));
```

```
    }

    @Test
    public void testCalculateGrade_Path4_NilaiC() {
        assertEquals("C", studentGrade.calculateGrade(65));
    }

    @Test
    public void testCalculateGrade_Path5_NilaiD() {
        assertEquals("D", studentGrade.calculateGrade(55));
    }

    @Test
    public void testCalculateGrade_Path6_NilaiE() {
        assertEquals("E", studentGrade.calculateGrade(30));
    }

    // ===== calculateGrade(): pengujian nilai batas =====

    @Test
    public void testCalculateGrade_NilaiBatas() {
        assertAll(
            () -> assertEquals("E",
studentGrade.calculateGrade(0)),
            () -> assertEquals("E",
studentGrade.calculateGrade(49)),
            () -> assertEquals("D",
studentGrade.calculateGrade(50)),
            () -> assertEquals("D",
studentGrade.calculateGrade(59)),
            () -> assertEquals("C",
studentGrade.calculateGrade(60)),
            () -> assertEquals("C",
studentGrade.calculateGrade(69)),
```

```
            () -> assertEquals("B",
studentGrade.calculateGrade(70)),
            () -> assertEquals("B",
studentGrade.calculateGrade(79)),
            () -> assertEquals("A",
studentGrade.calculateGrade(80)),
            () -> assertEquals("A",
studentGrade.calculateGrade(100))
        );
    }

    // ===== isPassed() =====

    @Test
    public void testIsPassed_Lulus() {
        assertTrue(studentGrade.isPassed(75));
    }

    @Test
    public void testIsPassed_BatasLulus60() {
        assertTrue(studentGrade.isPassed(60));
    }

    @Test
    public void testIsPassed_Skor59TidakLulus() {
        assertFalse(studentGrade.isPassed(59));
    }

    @Test
    public void testIsPassed_NilaiMinMax() {
        assertAll(
            () -> assertFalse(studentGrade.isPassed(0)),
            () -> assertTrue(studentGrade.isPassed(100))
        );
```

```
    }

    @Test
    public void testIsPassed_ScoreNegatif() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.isPassed(-1));
    }

    @Test
    public void testIsPassed_ScoreLebihDari100() {
        assertThrows(IllegalArgumentException.class,
                () -> studentGrade.isPassed(101));
    }
}
```

Jumlah method test: 8 untuk `calculateGrade()` (7 test path dan 1 test nilai batas) dan 6 untuk `isPassed()`, seluruhnya menggunakan `assertEquals`, `assertTrue`, `assertFalse`, `assertThrows`, dan `assertAll` sesuai materi JUnit 5. Berdasarkan analisis path di atas, test tersebut menyentuh seluruh cabang pada kedua method, sehingga coverage diharapkan tinggi. Angka sebenarnya dibaca dari laporan JaCoCo setelah `mvn test` dijalankan.

BAGIAN C: GITHUB ACTIONS

Soal 5

1. Nama workflow

Nama workflow ditentukan oleh key `name` di baris pertama, yaitu "Python CI". Nama inilah yang tampil di tab Actions pada GitHub.

2. Trigger/event yang memicu workflow

Trigger didefinisikan pada bagian `on:`, ada dua event:

1. push ke branch `main` atau `develop`, yaitu setiap ada commit yang di-push ke salah satu dari dua branch tersebut.
2. pull_request yang menargetkan branch `main`, yaitu ketika pull request dibuat atau diperbarui dengan tujuan merge ke main.

Push ke branch lain, misalnya `feature-login`, tidak akan menjalankan workflow ini kecuali melalui pull request ke main.

3. Fungsi strategy matrix

Bagian `strategy.matrix.python-version: ['3.10', '3.11', '3.12']` membuat GitHub menjalankan job `test` sebanyak tiga kali secara paralel, masing-masing dengan satu versi Python. Nilai versinya dipanggil lewat `${{ matrix.python-version }}` pada step Setup Python. Fungsinya adalah:

- Memastikan kode dan test kompatibel dengan beberapa versi Python sekaligus (3.10, 3.11, dan 3.12) dalam satu kali push.
- Menghemat penulisan karena tidak perlu membuat tiga job terpisah dengan isi yang hampir sama.
- Karena berjalan paralel, waktu total lebih singkat dibanding menjalankan satu per satu.
- Jika salah satu gagal, terlihat jelas pada versi Python mana masalahnya muncul.

4. Steps di dalam job test

Soal meminta 5 steps, tetapi pada YAML yang tertera di soal, job `test` hanya memiliki 4 step. Keempat step tersebut adalah:

No	Nama step	Fungsi
1	Checkout code	Mengambil source code repository ke runner dengan `actions/checkout@v4`.
2	Setup Python ${{ matrix.python-version }}	Memasang versi Python sesuai nilai matrix dengan `actions/setup-python@v5`.
3	Install dependencies	Meng-upgrade pip, lalu memasang `pytest` dan `pytest-cov` (`run`).
4	Run tests	Menjalankan `pytest --cov=src --cov-report=xml` untuk eksekusi test sekaligus mengukur coverage.

Pada contoh workflow PyTest di materi pertemuan 4 (`python-test.yml`), terdapat satu step tambahan di urutan kelima, yaitu Upload coverage report (`actions/upload-artifact@v4`) yang menyimpan hasil coverage sebagai artifact. Jika soal dimaksudkan mengacu pada versi lengkap tersebut, maka step kelimanya adalah step ini. Pada YAML di soal sendiri, step tersebut tidak ada.

Soal 6

Workflow disimpan pada file `.github/workflows/java-test.yml` dengan isi sebagai berikut.

.github/workflows/java-test.yml

```
name: Java Unit Test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
```

```
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report

      - name: Upload jacoco-report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Penjelasan:

- `name: Java Unit Test` sesuai ketentuan nama workflow.
- Trigger `push` ke `main` dan `develop`, serta `pull_request` ke `main`.
- `runs-on: ubuntu-latest` sebagai runner.
- Lima step berurutan: checkout, setup JDK 17 dengan `actions/setup-java@v4` (distribusi Temurin), `mvn test`, `mvn jacoco:report`, lalu upload artifact bernama **jacoco-report** dari folder `target/site/jacoco/` (lokasi keluaran laporan JaCoCo).

Agar `mvn jacoco:report` bisa menghasilkan laporan, plugin `jacoco-maven-plugin` harus sudah dikonfigurasi di `pom.xml` dengan goal `prepare-agent` dan `report` seperti pada materi. Tanpa `prepare-agent`, file `jacoco.exec` tidak terbentuk dan laporan tidak dapat dibuat.

BAGIAN D: CODE COVERAGE & INTEGRASI

Soal 7

1. Total statement di project

Kolom Stmts dijumlahkan: 10 + 15 + 20 + 5 = 50 statement, sesuai dengan baris TOTAL pada laporan.

2. Total statement yang tidak tereksekusi

Kolom Miss dijumlahkan: 0 + 2 + 8 + 1 = 11 statement. Jika diperiksa, coverage total = (50 − 11) / 50 = 39 / 50 = 78%, cocok dengan angka di laporan.

3. File dengan coverage paling rendah

File dengan coverage terendah adalah src/payment.py dengan coverage 60% (12 dari 20 statement tereksekusi, 8 statement belum). File ini juga menyumbang 8 dari 11 statement yang belum teruji, atau sekitar 73% dari seluruh kekurangan coverage di project.

4. Cara meningkatkan coverage payment.py

1. Cari baris yang belum tereksekusi. Jalankan `pytest --cov=src --cov-report=term-missing` atau gunakan laporan HTML (`--cov-report=html`) untuk melihat nomor baris dari 8 statement yang masih Miss.
2. Analisis penyebabnya dengan teknik white-box dari Minggu 3, yaitu dengan menggambar flow graph dan menghitung cyclomatic complexity `payment.py`. Biasanya baris yang terlewat berupa cabang `if/else` yang belum pernah dipenuhi, penanganan error/exception, atau kasus khusus (misalnya saldo tidak cukup, metode pembayaran tidak valid, nominal nol atau negatif).
3. Tambahkan test case untuk setiap independent path yang belum tercakup, baik skenario normal maupun skenario gagal. Gunakan `pytest.raises` untuk menguji exception dan `@pytest.mark.parametrize` bila banyak variasi input. Dependensi eksternal seperti gateway pembayaran sebaiknya di-mock.
4. Pastikan test benar-benar memakai assert pada hasil yang diharapkan agar peningkatan coverage ini bermakna, bukan sekadar menjalankan kode.
5. Jalankan ulang dan pantau pipeline di GitHub Actions sampai coverage payment.py naik. Dari 20 statement, minimal 16 harus tereksekusi agar mencapai 80%, artinya perlu mencakup sedikitnya 4 dari 8 statement yang terlewat. Dengan begitu total project naik

dari 78% menjadi sekitar 86% (7 miss dari 50), dan bila payment.py mencapai 100% total menjadi 94% (3 miss dari 50).

5. Apakah coverage 78% sudah cukup baik?

Menurut saya belum cukup baik, karena beberapa alasan:

- Target pada materi untuk statement dan line coverage adalah 80–90%, sedangkan 78% masih di bawah batas minimum. Pada konfigurasi JaCoCo di materi, aturan `check` dengan minimum 0,80 akan membuat build gagal bila coverage di bawahnya.
- Angka total menutupi sebaran yang tidak merata. `calculator.py` sudah 100%, tetapi `payment.py` baru 60%. Modul pembayaran biasanya paling kritis karena menyangkut uang sehingga risiko bug yang tidak terdeteksi paling besar justru di sana.
- Sebaliknya, ada juga yang perlu dicatat: coverage hanya mengukur seberapa banyak kode dijalankan, bukan seberapa bagus pengujiannya. Kode yang tereksekusi belum tentu terverifikasi bila assertion-nya lemah. Jadi 100% pun tidak menjamin bebas bug, dan angka 78% ini sebaiknya dipakai sebagai petunjuk area yang perlu diperbaiki, bukan satu-satunya ukuran kualitas.

Kesimpulannya, 78% sudah menunjukkan bahwa pengujian berjalan cukup baik, tetapi masih perlu ditingkatkan terutama pada `payment.py` sampai total mencapai minimal 80%, lebih baik 85–90%.

BAGIAN E: STUDI KASUS TERPADU

Soal 8

Project yang dibuat adalah bank-account-t**est** berbasis Java 17, Maven, JUnit 5, JaCoCo, dan GitHub Actions. Struktur direktorinya mengikuti konvensi dari materi:

```
bank-account-test/
├── .github/
│   └── workflows/
│       └── test.yml
├── src/
│   ├── main/java/com/stmik/BankAccount.java
│   └── test/java/com/stmik/BankAccountTest.java
├── .gitignore
└── pom.xml
```

1. Kode program: BankAccount.java

Aturan validasi: `deposit` dan `withdraw` menolak jumlah negatif, dan `withdraw` menolak jumlah yang melebihi saldo, keduanya dengan `IllegalArgumentException`. Saya juga menambahkan validasi saldo awal agar objek tidak bisa dibuat dalam kondisi tidak valid.

src/main/java/com/stmik/BankAccount.java

```
package com.stmik;

public class BankAccount {

    private double balance;

    public BankAccount() {
        this(0);
    }

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) {
```

```
            throw new IllegalArgumentException(
                    "Saldo awal tidak boleh negatif");
        }
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException(
                    "Jumlah deposit tidak boleh negatif");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException(
                    "Jumlah penarikan tidak boleh negatif");
        }
        if (amount > balance) {
            throw new IllegalArgumentException(
                    "Saldo tidak mencukupi");
        }
        balance -= amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

2. Unit test: BankAccountTest.java (9 test case)

Test mencakup kondisi normal, nilai batas (deposit nol, withdraw tepat sebesar saldo), dan kondisi error. Berdasarkan analisis, 9 test ini menyentuh semua cabang `BankAccount` (angka coverage aktual dilihat dari laporan JaCoCo). Jumlahnya melebihi minimal 6 test case.

src/test/java/com/stmik/BankAccountTest.java

```
package com.stmik;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class BankAccountTest {

    private BankAccount account;

    @BeforeEach
    public void setUp() {
        account = new BankAccount(100.0);   // saldo awal 100
    }

    @Test
    public void testSaldoAwalDefaultNol() {
        BankAccount baru = new BankAccount();
        assertEquals(0.0, baru.getBalance(), 0.001);
    }

    @Test
    public void testSaldoAwalNegatifMelemparException() {
        assertThrows(IllegalArgumentException.class,
                () -> new BankAccount(-1));
    }

    @Test
```

```
    public void testDepositMenambahSaldo() {
        account.deposit(50.0);
        assertEquals(150.0, account.getBalance(), 0.001);
    }

    @Test
    public void testDepositNolTidakMengubahSaldo() {
        account.deposit(0);
        assertEquals(100.0, account.getBalance(), 0.001);
    }

    @Test
    public void testDepositNegatifMelemparException() {
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> account.deposit(-10.0));
        assertEquals("Jumlah deposit tidak boleh negatif",
ex.getMessage());
    }

    @Test
    public void testWithdrawMengurangiSaldo() {
        account.withdraw(40.0);
        assertEquals(60.0, account.getBalance(), 0.001);
    }

    @Test
    public void testWithdrawSeluruhSaldo() {
        account.withdraw(100.0);          // tepat sama
dengan saldo
        assertEquals(0.0, account.getBalance(), 0.001);
    }

    @Test
```

```
    public void testWithdrawNegatifMelemparException() {
        assertThrows(IllegalArgumentException.class,
                () -> account.withdraw(-5.0));
    }

    @Test
    public void testWithdrawMelebihiSaldoMelemparException()
{
        IllegalArgumentException ex = assertThrows(
                IllegalArgumentException.class,
                () -> account.withdraw(500.0));
        assertEquals("Saldo tidak mencukupi",
ex.getMessage());
        assertEquals(100.0, account.getBalance(), 0.001); //
saldo tidak berubah
    }
}
```

No	Test case	Yang diverifikasi
1	testSaldoAwalDefaultNol	Konstruktor tanpa parameter, saldo 0
2	testSaldoAwalNegatifMelemparException	Validasi saldo awal negatif
3	testDepositMenambahSaldo	100 + 50 = 150
4	testDepositNolTidakMengubahSaldo	Nilai batas deposit 0
5	testDepositNegatifMelemparException	Deposit negatif ditolak + pesan error
6	testWithdrawMengurangiSaldo	100 − 40 = 60
7	testWithdrawSeluruhSaldo	Nilai batas: withdraw = saldo, hasil 0
8	testWithdrawNegatifMelemparException	Withdraw negatif ditolak

No	Test case	Yang diverifikasi
9	testWithdrawMelebihiSaldoMelemparException	Saldo tidak cukup ditolak dan saldo tetap

3. GitHub Actions: workflow dan konfigurasi pom.xml

.github/workflows/test.yml

```
name: Java CI BankAccount

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run unit tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco:report
```

```
      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Plugin JaCoCo dan JUnit 5 dikonfigurasi pada `pom.xml` mengikuti snippet di materi (goal `prepare-agent` dan `report`).

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.stmik</groupId>
  <artifactId>bank-account-test</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
```

```
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

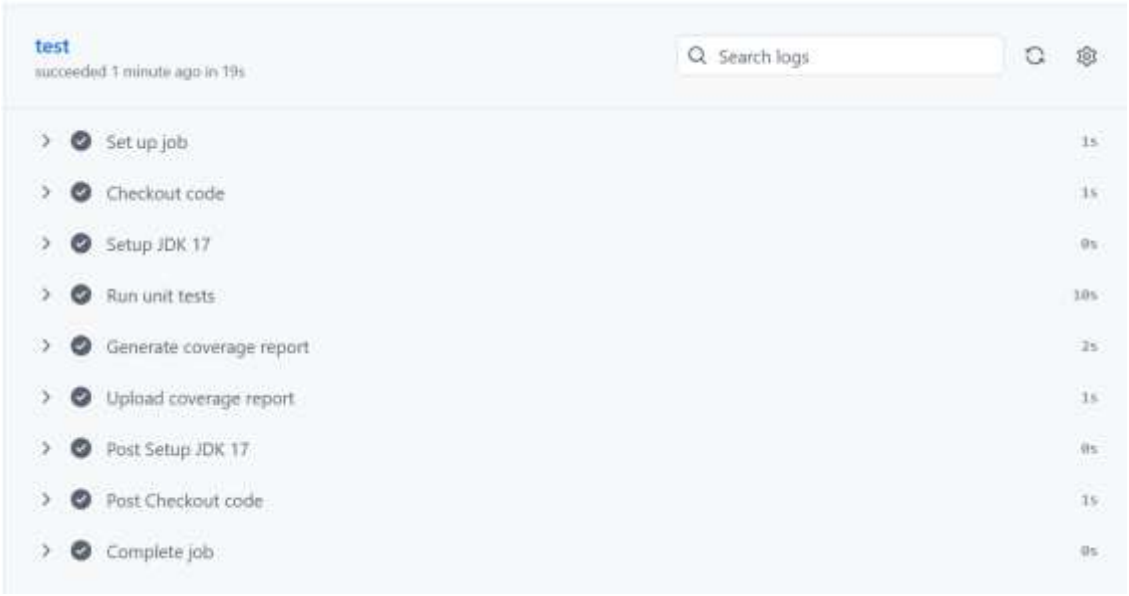
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.jacoco</groupId>
        <artifactId>jacoco-maven-plugin</artifactId>
        <version>0.8.11</version>
        <executions>
          <execution>
            <id>prepare-agent</id>
            <goals><goal>prepare-agent</goal></goals>
          </execution>
          <execution>
            <id>report</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
</project>
```

4. Repository dan bukti pipeline

Langkah upload: buat repository di GitHub, push seluruh project dengan `git add .`, `git commit -m "feat: add BankAccount with unit tests"`, dan `git push origin main`. Setelah itu buka tab Actions untuk memastikan workflow berstatus sukses, lalu unduh artifact jacoco-report untuk melihat coverage.

Isi berikut diisi setelah project di-push dan pipeline berhasil dijalankan:

- Link repository: https://github.com/MelaniePriatna24/bank-account-test.git
- Screenshot pipeline sukses:



- Screenshot coverage report:



LINK REPOSITORY GITHUB

Seluruh pekerjaan diunggah ke GitHub sesuai ketentuan (folder `Tugas/Tugas04`).

Item	Link
Repository tugas	https://github.com/MelaniePriatna24/Tugas-Melanie.git
Repository latihan Git (Soal 2)	https://github.com/MelaniePriatna24/latihan-git-3224015.git
Repository BankAccount (Soal 8)	https://github.com/MelaniePriatna24/bank-account-test.git